SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

CO1-ASSESSMENT TOOL 1 – APPLICATION BASED QUESTIONS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – Application Based Questions
Weightage:	40% of CIA		
CO1 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	SHAM S	**Reg. No.:**	192325076
Section / Batch:	AI & ML	**Date:**	10/10/2026

Code of Conduct

I SHAM S. (192325076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Information Extraction	Regex-Based Information Extraction	1
Pattern Matching	Regex Pattern Matching	2
Regular Expression Patterns	Regex-Based Input Validation	3

Annexure A – Application Based Questions

Section 1: Regular Expressions – Resume Information Extraction

1. A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.
Identify email addresses.
Extract mobile numbers.
Detect technical skills (Python, Java, SQL, Machine Learning, NLP).
Extract years of experience.
Generate a structured summary of the candidate's profile.
Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Section 2: Regular Expressions – Product Search System

2. An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular Expressions that performs the following tasks:

Search products using an exact keyword.
Perform prefix-based searches.
Perform suffix-based searches.
Search using partial keywords.
Perform case-insensitive searches.
Display all matching product names.
Generate a report showing the total number of matching products for each search.

Section 3: Regular Expressions – University Registration System

A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.
Validate the institutional email address.
Validate the course code.
Validate the semester information.
Validate the student's mobile number.

Display appropriate validation messages for each field.
Generate a final registration status report indicating whether the registration is successful.

Question 1: Resume Information Extraction Using Regular Expressions

Aim

To develop a Python program using Regular Expressions to extract candidate details, including name, email, mobile number, technical skills, and years of experience from a resume.

Algorithm

1. Import the re module.
2. Store the resume content as a string.
3. Extract the candidate's name, email address, and mobile number using regular expressions.
4. Identify technical skills such as Python, Java, SQL, Machine Learning, and NLP.
5. Extract years of experience.
6. Display the candidate's profile as a structured summary.
7. Check whether the candidate has at least 2 years of experience and Python skill.

Python Program

```
import re


resume = """
Name: Rahul Kumar
Email: rahul@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning, NLP
Experience: 3 years
"""


# Extract candidate name
name = re.search(r"Name:\s*(.+)", resume)
name = name.group(1).strip() if name else "Not found"


# Extract email
email = re.search(r"[\w.-]+@[\w.-]+\.\w+", resume)
email = email.group() if email else "Not found"


# Extract mobile number
```

```
mobile = re.search(r"\b[6-9]\d{9}\b", resume)
mobile = mobile.group() if mobile else "Not found"

# Detect technical skills
skills_list = [
    "Python", "Java", "SQL",
    "Machine Learning", "NLP"
]

skills = [
    skill for skill in skills_list
    if re.search(r"\b" + re.escape(skill) + r"\b",
                 resume, re.IGNORECASE)
]

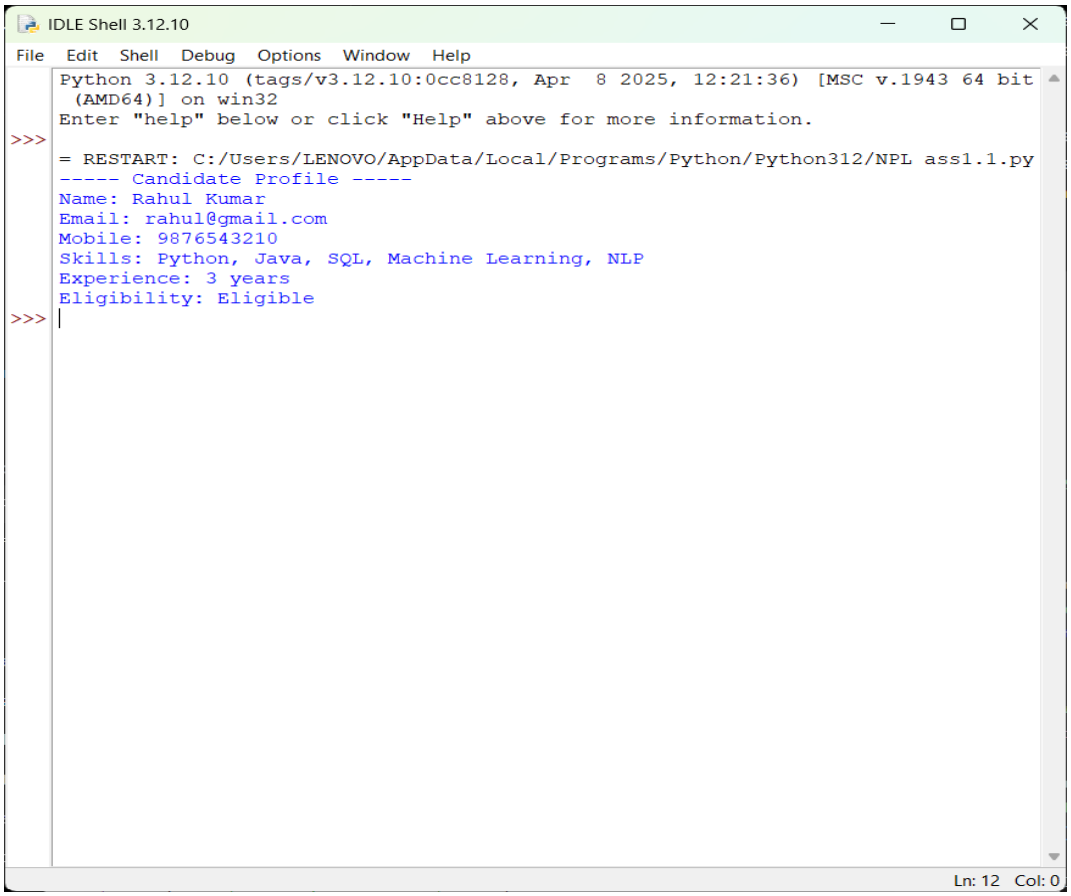
# Extract years of experience
experience = re.search(
    r"Experience:\s*(\d+)\s*years?",
    resume,
    re.IGNORECASE
)

years = int(experience.group(1)) if experience else 0

# Display candidate profile
print("----- Candidate Profile -----")
print("Name:", name)
print("Email:", email)
print("Mobile:", mobile)
print("Skills:", ", ".join(skills))
print("Experience:", years, "years")
```

```
# Check eligibility
if years >= 2 and any(
    skill.lower() == "python" for skill in skills
):
    print("Eligibility: Eligible")
else:
    print("Eligibility: Not Eligible")
```

Output



Explanation

- re.search() searches for a matching pattern in the resume.
- \w matches word characters.
- \d matches digits.
- \s* matches zero or more whitespace characters.
- re.IGNORECASE allows case-insensitive matching.
- The candidate is eligible if they have at least 2 years of experience and Python skill.

Result

Thus, the Python program successfully extracts resume information using Regular Expressions and identifies candidates who meet the eligibility criteria.

Question 2: Product Search System Using Regular Expressions

Aim

To develop a Python program using Regular Expressions to search products by exact keyword, prefix, suffix, partial keyword, and case-insensitive keyword.

Algorithm

1. Import the re module.
2. Store product names in a list.
3. Accept a search keyword and search type.
4. Apply the appropriate regular expression.
5. Display all matching product names.
6. Count and report the number of matching products.

Python Program

```
import re


products = [
    "Apple iPhone",
    "Samsung Galaxy",
    "Apple Watch",
    "Wireless Headphones",
    "Laptop Computer",
    "Apple MacBook",
    "Bluetooth Speaker"
]


def search_products(keyword, search_type):
    if search_type == "exact":
        pattern = r"^" + re.escape(keyword) + r"$"


    elif search_type == "prefix":
```

```
        pattern = r"^" + re.escape(keyword)

    elif search_type == "suffix":
        pattern = re.escape(keyword) + r"$"

    elif search_type == "partial":
        pattern = re.escape(keyword)

    else:
        print("Invalid search type")
        return

    matches = [
        product for product in products
        if re.search(pattern, product, re.IGNORECASE)
    ]

    print("\nSearch Type:", search_type)
    print("Keyword:", keyword)
    print("Matching Products:")

    if matches:
        for product in matches:
            print("-", product)
    else:
        print("No products found")

    print("Total Matches:", len(matches))


# Perform different searches
search_products("Apple Watch", "exact")
```

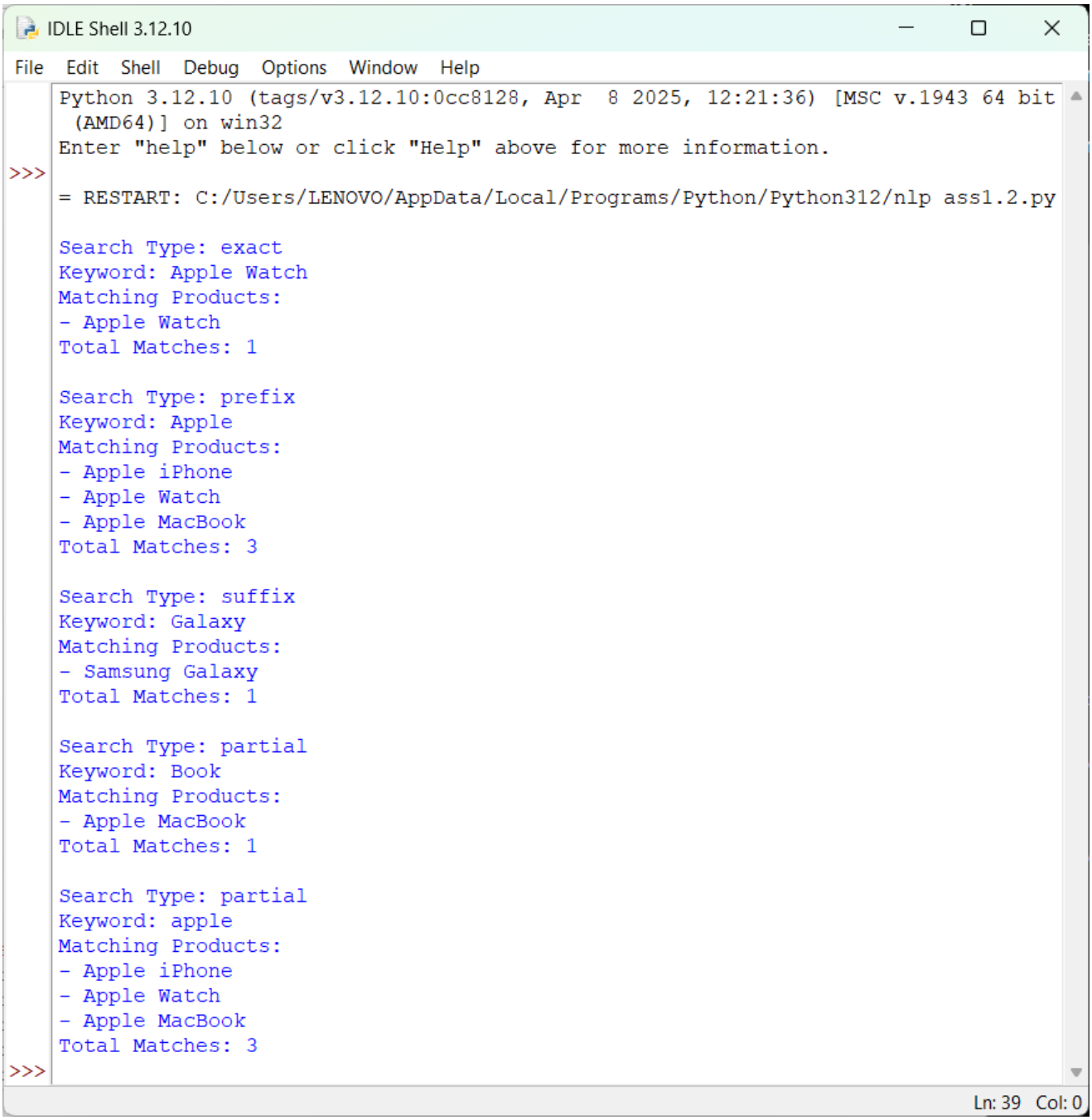
search_products("Apple", "prefix")

search_products("Galaxy", "suffix")

search_products("Book", "partial")

search_products("apple", "partial")

Output



Explanation

The program supports five search requirements:

Search Type	Regular Expression	Purpose
Exact	^keyword$	Matches the entire product name
Prefix	^keyword	Matches products beginning with the keyword
Suffix	keyword$	Matches products ending with the keyword
Partial	keyword	Matches the keyword anywhere in the name
Case-insensitive	re.IGNORECASE	Matches regardless of uppercase or lowercase letters

Note: re.escape() ensures that special characters in a search keyword are treated as ordinary characters rather than regex operators.

Result

Thus, the Python program successfully implements flexible product searches using Regular Expressions and displays the matching products along with their total count.

Question 3: University Registration System Using Regular Expressions

Aim

To develop a Python program using Regular Expressions to validate student registration details, including register number, institutional email, course code, semester, and mobile number.

Assumption: The question does not specify the university's exact formats. Therefore, this example uses illustrative formats that can be modified according to the institution's rules.

Validation Rules

Field	Validation Rule	Example
Register Number	12 digits	123456789012
Institutional Email	Must end with @simats.edu.in	student@simats.edu.in
Course Code	Three uppercase letters followed by three digits	NLP101
Semester	Integer from 1 to 8	5
Mobile Number	Indian 10-digit number starting with 6–9	9876543210

Algorithm

1. Import the re module.
2. Accept the student's registration details.
3. Define regular expression patterns for each field.
4. Validate each input using re.fullmatch().

5. Display a suitable message for every field.
6. Generate the final registration status.

Python Program

```
import re

# Input student details
register_no = input("Enter Register Number: ")
email = input("Enter Institutional Email: ")
course_code = input("Enter Course Code: ")
semester = input("Enter Semester: ")
mobile = input("Enter Mobile Number: ")

# Validation patterns
register_pattern = r"\d{12}"
email_pattern = r"[\w.-]+@simats\.edu\.in"
course_pattern = r"[A-Z]{3}\d{3}"
mobile_pattern = r"[6-9]\d{9}"

# Validate register number
valid_register = bool(
    re.fullmatch(register_pattern, register_no)
)
print("Register Number:",
      "Valid" if valid_register else "Invalid")

# Validate institutional email
valid_email = bool(
    re.fullmatch(email_pattern, email)
)
print("Institutional Email:",
      "Valid" if valid_email else "Invalid")
```

```
# Validate course code
valid_course = bool(
    re.fullmatch(course_pattern, course_code)
)
print("Course Code:",
      "Valid" if valid_course else "Invalid")

# Validate semester
valid_semester = (
    semester.isdigit() and 1 <= int(semester) <= 8
)
print("Semester:",
      "Valid" if valid_semester else "Invalid")

# Validate mobile number
valid_mobile = bool(
    re.fullmatch(mobile_pattern, mobile)
)
print("Mobile Number:",
      "Valid" if valid_mobile else "Invalid")

# Final registration status
if all([
    valid_register,
    valid_email,
    valid_course,
    valid_semester,
    valid_mobile
]):
    print("\nRegistration Status: Successful")
else:
    print("\nRegistration Status: Failed")
```

Sample Input

Enter Register Number: 123456789012

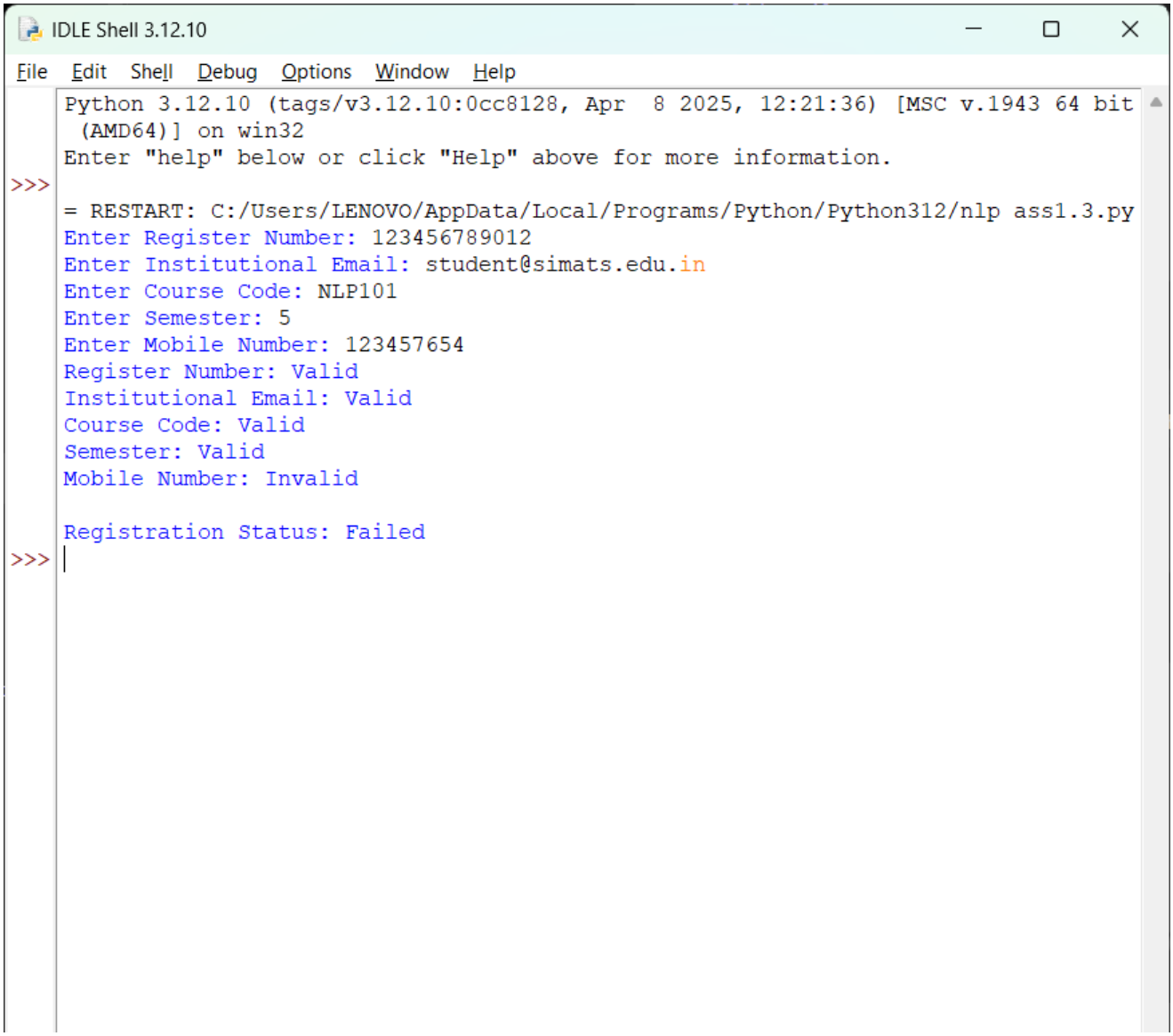
Enter Institutional Email: student@simats.edu.in

Enter Course Code: NLP101

Enter Semester: 5

Enter Mobile Number: 9876543210

Sample Output



Registration Status: Successful

Explanation

- re.fullmatch() checks whether the entire input matches the specified pattern.
- \d{12} accepts exactly 12 digits.

- [A-Z]{3}\d{3} accepts three uppercase letters followed by three digits.
- [6-9]\d{9} accepts a 10-digit Indian mobile number beginning with 6, 7, 8, or 9.
- all() returns True only when every validation condition is satisfied.
- Registration succeeds only if all five fields are valid.

Result

Thus, the Python program successfully validates student registration details using Regular Expressions and generates the final registration status.

Important Regular Expression Symbols for Your Assessment

Symbol	Meaning	Example
\d	Matches a digit	\d{10}
\w	Matches a word character	\w+
\s	Matches whitespace	\s*
.	Matches any character except newline by default	a.c
^	Beginning of a string	^Hello
$	End of a string	world$
+	One or more occurrences	\d+
*	Zero or more occurrences	\s*
{n}	Exactly n occurrences	\d{10}
[A-Z]	Any uppercase English letter	[A-Z]+
re.search()	Finds a matching pattern within a string	Email extraction
re.fullmatch()	Checks the entire string against a pattern	Input validation
re.IGNORECASE	Ignores letter case	Searching for apple

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, wellstructured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, wellorganized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Q. No. / Criteria	Understanding of Problem Context	Application of Concepts	Problem-Solving Approach	Accuracy of Solution	Clarity	Sub. Total	Total %
1							
2							
3							